# EST-744

## Workshop Tensorflow

Juan Zamora O.

Deep Learning






# Workshop Objectives (90 minutes)

By the end of this workshop, you will:

- Build neural networks using **all three Keras APIs**
- Know when to use Sequential, Functional, or Model Subclassing
- Train models and interpret results
- Debug and improve model performance
- Deploy models for inference

**Three ways to build a model in Keras**:

```
Sequential    → simple, linear pipelines
Functional    → branching, multi-input/output architectures
Subclassing   → fully custom, research-grade control
```

# Environment Setup

## Check Your Installation

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print(f"TensorFlow version: {tf.__version__}")
print(f"Keras version: {tf.keras.__version__}")

# GPU check (optional)
print(f"GPU available: {len(tf.config.list_physical_devices('GPU')) > 0}")
```

**If missing packages:**

```
pip install tensorflow numpy pandas matplotlib scikit-learn
```

# Topic 1.1: Sequential API (The Simple Way)

**Concept**: Stack layers in order (input → hidden → output)

## Code Block 1.1a: Create a Simple Network

```
from tensorflow import keras
from tensorflow.keras import layers

# Method 1: Sequential API (simplest)
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Method 2: Add layers incrementally
model = keras.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10,)))
model.add(layers.Dense(32, activation='relu'))
```

# Topic 1.2: Compile: Loss, Optimizer, Metrics

**Concept**: Tell the model how to learn and what to measure

## Code Block 1.2a: Compile a Network

```
model.compile(
    loss='binary_crossentropy',      # What to minimize
    optimizer='adam',                # How to learn
    metrics=['accuracy']             # What to track
)
```

**Common Loss Functions:** - `binary_crossentropy`: Binary classification (2 classes) - `categorical_crossentropy`: Multi-class (3+ classes) - `sparse_categorical_crossentropy`: Multi-class (integer labels) - `mse`: Regression (continuous values)

**Common Optimizers:** - `adam`: Adaptive, most popular - `sgd`: Stochastic gradient descent - `rmsprop`: Root mean square propagation

**Common Metrics:** - `accuracy`: Classification accuracy - `mse`: Mean squared error - `mae`: Mean absolute error - `auc`: Area under ROC curve

# Topic 1.3: Train: Fit the Model

**Concept**: Feed data to the model, let it learn

## Code Block 1.3a: Prepare Data

```
# Generate dummy data for binary classification
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = make_classification(
    n_samples=1000,      # 1000 examples
    n_features=10,       # 10 input features
    n_classes=2,         # Binary classification
    random_state=42
)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

# Topic 1.4: Evaluate: Test the Model

**Concept**: See how well it generalizes to unseen data

## Code Block 1.4a: Evaluate on Test Set

```
# Get loss and metrics on test data
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy:.4f}")

# Make predictions
y_pred_prob = model.predict(X_test[:5])  # Probabilities
y_pred = (y_pred_prob > 0.5).astype(int).flatten()  # Binary predictions

print(f"First 5 predictions: {y_pred}")
print(f"First 5 actual: {y_test[:5]}")
```

# Topic 1.5: Visualize Training

**Concept**: Understand what happened during training

## Code Block 1.5a: Plot Loss and Accuracy

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

# Plot 1: Loss
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss Over Time')
plt.legend()
plt.grid(True)
```

# Topic 2.1: Iris Dataset (Classification)

**Concept**: Multi-class classification with real data

## Code Block 2.1a: Load and Explore Data

```
from tensorflow.keras.datasets import fashion_mnist
import numpy as np

# Load Iris dataset
from sklearn.datasets import load_iris
iris = load_iris()

X = iris.data  # 150 samples, 4 features
y = iris.target  # 3 classes (0, 1, 2)

print(f"Features: {X.shape}")
print(f"Classes: {np.unique(y)}")
print(f"Class distribution: {np.bincount(y)}")
```

# Topic 2.2: MNIST Digits (Image Classification)

**Concept**: Neural networks on images (preprocessing included)

## Code Block 2.2a: Load and Preprocess MNIST

```
from tensorflow.keras.datasets import mnist

# Load dataset (auto-downloads ~11 MB)
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print(f"Training images: {X_train.shape}")  # (60000, 28, 28)
print(f"Training labels: {y_train.shape}")  # (60000,)

# Images are 28×28 pixels, flatten to 784
X_train_flat = X_train.reshape(-1, 784).astype('float32') / 255.0
X_test_flat = X_test.reshape(-1, 784).astype('float32') / 255.0

# One-hot encode labels
y_train_cat = keras.utils.to_categorical(y_train, 10)
```

# Topic 2.3: Understand Dropout

**Concept**: Regularization technique to prevent overfitting

## Code Block 2.3a: With vs Without Dropout

```
# Model WITHOUT dropout
model_no_dropout = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model_no_dropout.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

history_no_dropout = model_no_dropout.fit(
```

# The Three Keras APIs: When to Use Each

All three APIs produce fully compatible `keras.Model` objects — same `compile()`, `fit()`, `evaluate()`, `predict()`, `save()`.

| API | Control | Verbosity | Best For |
|---|---|---|---|
| **Sequential** | Low | Minimal | Linear stacks, quick prototyping |
| **Functional** | Medium | Moderate | Multi-input, skip connections, branching |
| **Subclassing** | Full | Verbose | Custom layers, research, dynamic graphs |

**Analogy**: Like Python classes vs functions vs scripts — each has its place.

# Topic 3.1: Functional API (Medium Control)

**Concept**: Build models as a directed graph of layers

## Code Block 3.1a: Functional API — Same Task, More Flexibility

```
from tensorflow import keras
from tensorflow.keras import layers

# Sequential (familiar)
model_seq = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Functional (same architecture, explicit data flow)
inputs = keras.Input(shape=(10,))
x = layers.Dense(64, activation='relu')(inputs)
x = layers.Dense(32, activation='relu')(x)
```

# Topic 3.2: Model Subclassing (Full Control)

**Concept**: Define a model as a Python class — maximum flexibility for custom behaviour

**When to prefer it**: - Layers whose behaviour changes between training and inference (e.g. custom dropout) - Dynamic computation graphs (variable number of steps, loops, conditions) - Research prototyping where every forward pass may differ

**Key rule**: Override `__init__` to ***define*** layers, `call` to ***use*** them.

# Topic 3.2 (cont.): Subclassing — Basic Pattern

## Code Block 3.2a: Minimum Working Example

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

class SimpleClassifier(keras.Model):
    """Equivalent to the Sequential model in Part 1."""

    def __init__(self):
        super().__init__()                          # ← always call super().__
        self.dense1 = layers.Dense(64, activation='relu')
        self.dense2 = layers.Dense(32, activation='relu')
        self.out    = layers.Dense(1,  activation='sigmoid')

    def call(self, x, training=False):              # ← forward pass logic live
        x = self.dense1(x)
        x = self.dense2(x)
```

# Topic 3.2 (cont.): Subclassing — Adding Dropout Correctly

## Code Block 3.2b: `training` Flag Controls Behaviour

```
class ClassifierWithDropout(keras.Model):
    """
    Dropout is ON during training, OFF during inference.
    The `training` argument in call() propagates this signal.
    """

    def __init__(self, dropout_rate=0.3):
        super().__init__()
        self.dense1   = layers.Dense(128, activation='relu')
        self.drop1    = layers.Dropout(dropout_rate)
        self.dense2   = layers.Dense(64,  activation='relu')
        self.drop2    = layers.Dropout(dropout_rate)
        self.out      = layers.Dense(10,  activation='softmax')

    def call(self, x, training=False):
        x = self.dense1(x)
```

# Topic 3.2 (cont.): Subclassing — Custom Layer Inside a Model

## Code Block 3.2c: Reusable Custom Layer

```
class ResidualBlock(keras.layers.Layer):
    """
    A simple residual (skip-connection) block.
    Adds the input directly to the output: out = F(x) + x.
    Used in ResNet-style architectures.
    """

    def __init__(self, units):
        super().__init__()
        self.dense1 = layers.Dense(units, activation='relu')
        self.dense2 = layers.Dense(units)          # no activation here
        self.add    = layers.Add()
        self.relu   = layers.Activation('relu')

    def call(self, x):
        residual = x                               # ← save original input
```

# Topic 3.2 (cont.): API Comparison Side-by-Side

## Code Block 3.2d: Same Network, Three Styles

```
#  Sequential
seq = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1,  activation='sigmoid')
])

#  Functional
inp  = keras.Input(shape=(10,))
x    = layers.Dense(64, activation='relu')(inp)
x    = layers.Dense(32, activation='relu')(x)
out  = layers.Dense(1,  activation='sigmoid')(x)
func = keras.Model(inp, out)

#  Subclassing
class SubModel(keras.Model):
```

# Topic 3.3: Custom Training Loop (GradientTape)

**Concept**: Full control over the training process

## Code Block 3.3a: Gradient Tape (TensorFlow's Autograd)

```
# Simple regression: y = 2x + noise
X = np.random.randn(100, 1).astype('float32')
y = 2 * X + np.random.randn(100, 1) * 0.1

# Build model
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(1,)),
    layers.Dense(1)
])

# Loss function and optimizer
loss_fn = keras.losses.MeanSquaredError()
optimizer = keras.optimizers.Adam(learning_rate=0.01)
```

# Topic 3.4: Callbacks (Monitor & Control Training)

**Concept**: Stop training early, save best model, adjust learning rate

## Code Block 3.4a: Early Stopping and Model Checkpoint

```
# Build model
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Create callbacks
```

# Topic 3.5: Regularization Techniques

**Concept**: Prevent overfitting by constraining weights

## Code Block 3.5a: L1/L2 Regularization

```
# Model WITHOUT regularization
model_no_reg = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(10,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Model WITH L2 regularization
model_l2 = keras.Sequential([
    layers.Dense(128, activation='relu',
                 input_shape=(10,),
                 kernel_regularizer=keras.regularizers.l2(0.001)),
    layers.Dense(64, activation='relu',
                 kernel_regularizer=keras.regularizers.l2(0.001)),
```

# Topic 4.1: Identify and Fix Common Problems

**Concept**: Diagnose why models fail to learn

## Code Block 4.1a: Common Issues and Solutions

```
# ISSUE 1: Loss is NaN or Inf
# Causes: exploding gradients, bad learning rate, invalid data

# Solution: Check data for NaN
print(f"NaN in X_train: {np.isnan(X_train).any()}")
print(f"NaN in y_train: {np.isnan(y_train).any()}")

# Solution: Normalize input
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Solution: Use gradient clipping
model = keras.Sequential([
```

# Topic 4.2: Hyperparameter Tuning

**Concept**: Find good hyperparameters systematically

## Code Block 4.2a: Grid Search with Keras

```
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier

# Define model builder function
def create_model(learning_rate=0.001, dropout_rate=0.2,
                 hidden_units=64):
    model = keras.Sequential([
        layers.Dense(hidden_units, activation='relu',
                     input_shape=(10,)),
        layers.Dropout(dropout_rate),
        layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
```

# Topic 4.3: Save and Load Models

**Concept**: Persist trained models for later use

## Code Block 4.3a: Model Persistence

```
# Build and train a model
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam',
              metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10, batch_size=32,
                    validation_split=0.2, verbose=0)

# Method 1: Save entire model (architecture + weights)
```

# Exercise 1: Build and Train (Beginner — 5 min)

**Task**: Create a binary classifier for synthetic data using Sequential API

```
# 1. Generate classification data
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=500, noise=0.1, random_state=42)

# 2. Split and normalize
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
X_train = StandardScaler().fit_transform(X_train)
X_test  = StandardScaler().fit_transform(X_test)

# 3. Build model with:
#    - 2 inputs (from 2D data)
#    - One hidden layer with 16 neurons + ReLU
#    - Output layer with sigmoid
#
# 4. Compile and train for 20 epochs
```

# Exercise 2: Model Subclassing (Medium — 15 min)

**Task**: Recreate the same binary classifier using a subclassed model, then extend it

```
# PART A: Basic subclassed model (5 min)
# Create a subclassed model equivalent to Exercise 1's Sequential model.
# Hint: define layers in __init__, use them in call().
# Compile, fit, and print test accuracy — should match Exercise 1.

# PART B: Add a configurable block (10 min)
# Extend your model with a configurable number of hidden layers.
# - __init__ should accept n_layers=2 and units=32 as arguments
# - Build that many Dense layers dynamically inside __init__
# - Apply them in sequence inside call()
# - Compare test accuracy for n_layers in [1, 2, 3]

class ConfigurableNet(keras.Model):
    def __init__(self, n_layers=2, units=32, dropout_rate=0.0):
        super().__init__()
```

# Exercise 3: API Comparison (Medium — 5 min)

**Task**: Build the same 64-32-1 network in all three styles, confirm identical accuracy

```
# 1. Build the SAME architecture three times:
#    Sequential, Functional, Subclassed
#
# 2. Train all three on make_moons data (same splits)
#
# 3. Create a summary table:
#    | API         | Test Accuracy | Trainable Params | Lines of Code |
#    |-------------|---------------|------------------|---------------|
#    | Sequential  | ?             | ?                | ?             |
#    | Functional  | ?             | ?                | ?             |
#    | Subclassing | ?             | ?                | ?             |
#
# 4. Answer:
#    a) Does the API choice affect accuracy?
#    b) Which requires the most/least code?
```

# Key Takeaways

1. **Sequential API**: Linear stacking — fast to write, limited flexibility
2. **Functional API**: Explicit graph — multi-input/output, skip connections
3. **Model Subclassing**: Full Python class — custom logic, dynamic graphs, research
4. **Training process**: `model.fit()` works identically across all three APIs
5. **Debugging**: Check data → learning rate → regularization
6. **Validation**: Always monitor training vs validation curves
7. **Persistence**: `model.save()` works for all three API styles

# Common Pitfalls to Avoid

```
# x Don't forget to normalize data
model.fit(X_raw, y)  # Bad!

# o Always scale inputs
X_scaled = StandardScaler().fit_transform(X)
model.fit(X_scaled, y)  # Good!

# x Don't use validation_split AND separate validation set
model.fit(X, y, validation_data=(X_val, y_val),
          validation_split=0.2)  # Confusing!

# o Use one or the other
model.fit(X, y, validation_split=0.2)  # Good!

# x Don't train on test set
history = model.fit(X_test, y_test, ...)  # Bad!
```

# Resources for Continued Learning

**Official Documentation:** - TensorFlow: https://www.tensorflow.org/guide - Keras: https://keras.io/ - API Reference: https://www.tensorflow.org/api_docs/python/keras

**Recommended Tutorials:** - Keras sequential guide: https://keras.io/getting_started/sequential_model/ - Functional API: https://keras.io/getting_started/functional/ - Training from scratch: https://keras.io/getting_started/intro_to_keras_for_engineers/

**Practice Datasets:** - MNIST, CIFAR-10: `tensorflow.keras.datasets` - UCI Machine Learning: https://archive.ics.uci.edu/ml/ - Kaggle: https://www.kaggle.com/

# Questions & Discussion

**What we covered in 90 minutes:**

- Keras **Sequential** API (build, compile, train)
- Keras **Functional** API (multi-input, branching)
- Keras **Model Subclassing** (full Python class, custom layers, residual blocks)
- The three APIs compared side-by-side
- Real datasets (Iris, MNIST, synthetic data)
- Debugging and optimization techniques
- Custom training loop, callbacks, regularization
- Hands-on exercises (3 progressively difficult)

**What to practice:**

- Implement a model you read about in a paper using Subclassing

# Quick Reference

## Sequential API

```
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dropout(0.2),
    layers.Dense(1, activation='sigmoid')
])
```

## Functional API

```
inputs  = keras.Input(shape=(10,))
x       = layers.Dense(64, activation='relu')(inputs)
x       = layers.Dropout(0.2)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)
model   = keras.Model(inputs, outputs)
```

## Model Subclassing

```
class MyModel(keras.Model):
    def __init__(self):
```

# End of Workshop

**Thank you for participating!**

All code is available in accompanying Jupyter notebook.

Remember: The best way to learn deep learning is by building models.

**Start coding!**